Offline AI-Based Accident Detection Mobile Application

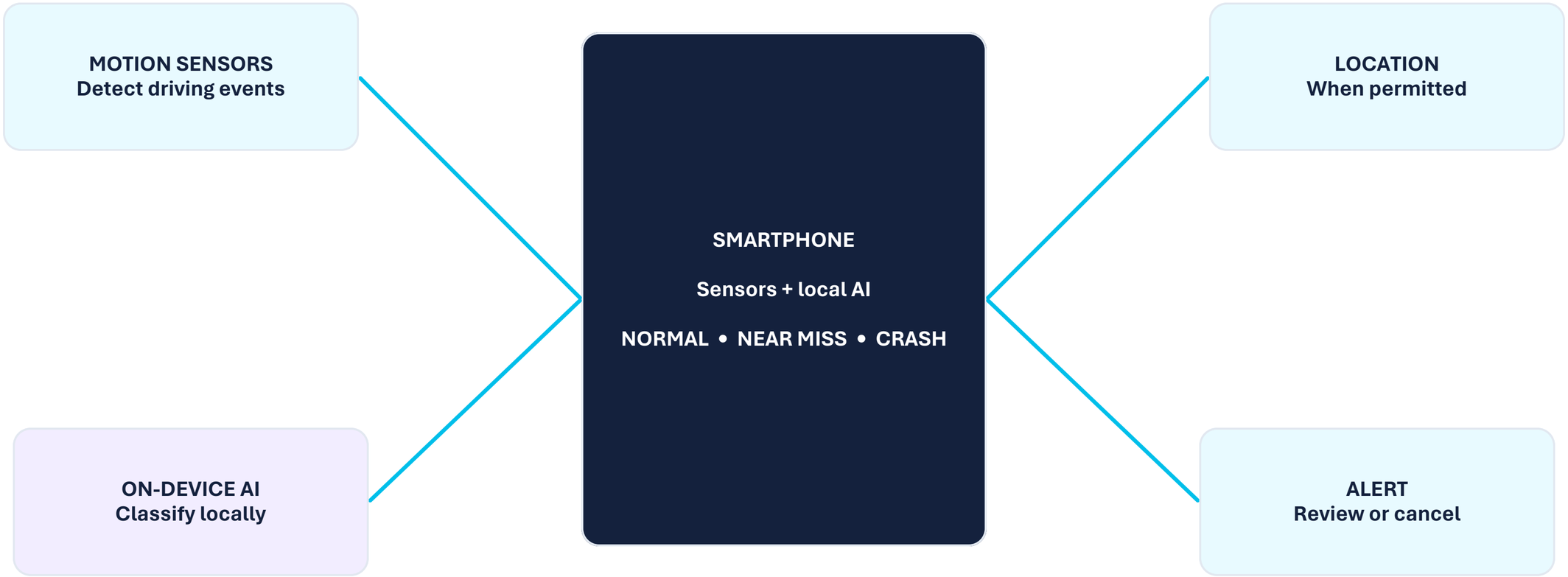
On-Device AI for Smarter Road Safety

TEAM MEMBERS • INSTITUTION • SUPERVISOR



The Idea in One Minute

A safety loop that stays on the phone.



Core detection operates fully offline; connectivity is not a prerequisite for classification.

The Problem We Want to Solve

01 Manual reporting may fail

A driver may be unable to report an incident.

02 Motion needs context

Sudden movement is difficult to interpret in isolation.

03 Connectivity is uncertain

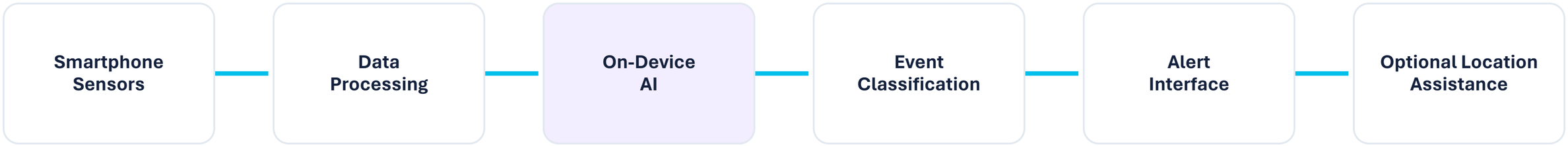
Internet may be unavailable or unreliable on the road.

04 False alarms erode trust

Automation must make review and cancellation clear.

Our Solution and Main Goals

A modular, offline-first path from motion to a user decision.



- Offline-first detection
- Lightweight mobile AI
- Three-class classification
- Alerts with cancellation
- Local event history
- Privacy-conscious processing

How the System Works

GPS supports the record; it is not an input to the classifier.



Core Application Features

Real-Time Sensor Monitoring

Collect motion data from supported sensors.

On-Device AI Classification

Analyze sensor patterns locally.

Three-Class Event Detection

Normal Driving, Near Miss, or Crash.

Crash Alert + Countdown

Possible-crash alert with cancellation.

GPS Location Capture

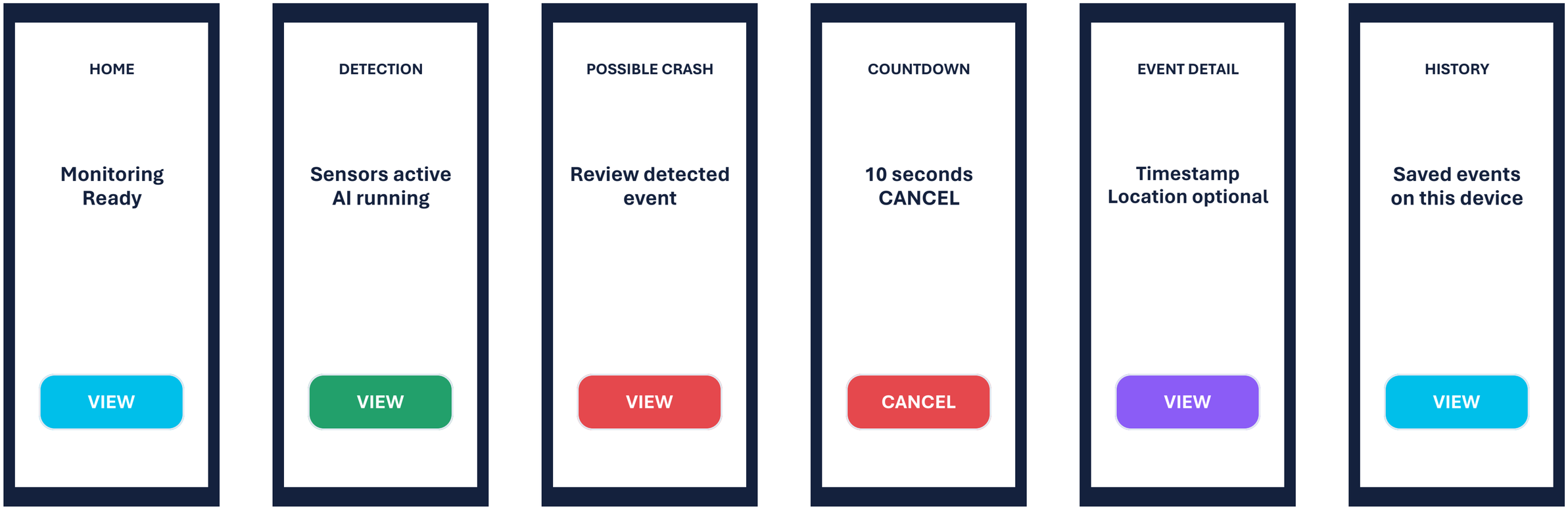
Attempt capture when permission is available.

Local Event History

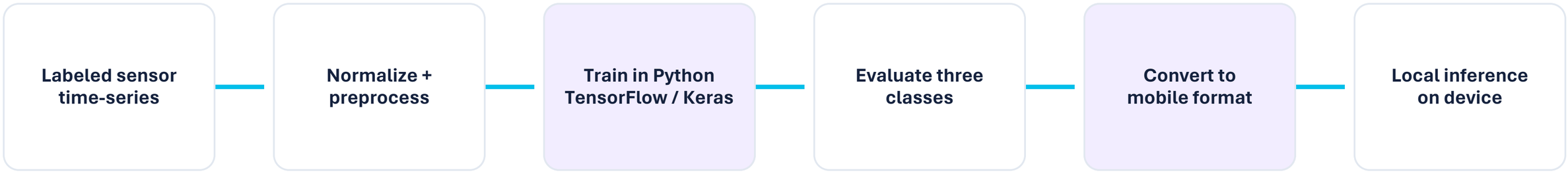
Save and display past detected events.

The User Experience

Proposed UI concepts — not necessarily implemented yet.



AI Model: From Sensor Data to Prediction



Three outputs: NORMAL DRIVING • NEAR MISS • CRASH

Pipeline overview only — no formulas or unsupported performance claims.

Development Roadmap

Proposed 10-week plan

W1–2

Requirements, research, system design

W3–4

Dataset prep + AI model

W5–6

Flutter UI + sensors

W7

On-device model

W8

Alerts, GPS, storage

W9

Integration + evaluation

W10

Improve, document, demo



Our Five-Person Team

Connected responsibilities; every member contributes to integration and testing.

Flutter UI + Navigation

Screens, navigation, reusable components

AI Model + Dataset

Data preparation, training, evaluation

Sensors, GPS + Android

Platform integration and permissions

Storage, Alerts + Integration

History and alert workflow

Testing + Documentation

QA, functional tests, technical writing

Roles may be adjusted as integration needs evolve.

What Existing Solutions Do

Five cautious, fact-based categories from the literature landscape.

Smartphone Sensor-Based

Embedded sensors; no extra hardware.
Challenge: sensor quality and context.

Connected Vehicle Systems

Vehicle/infrastructure data sharing.
Challenge: network and integration.

Cloud Monitoring

Remote processing can scale.
Challenge: connectivity and privacy.

Wearable / Dedicated Hardware

Tailored impact sensors.
Challenge: cost and adoption.

On-Device ML

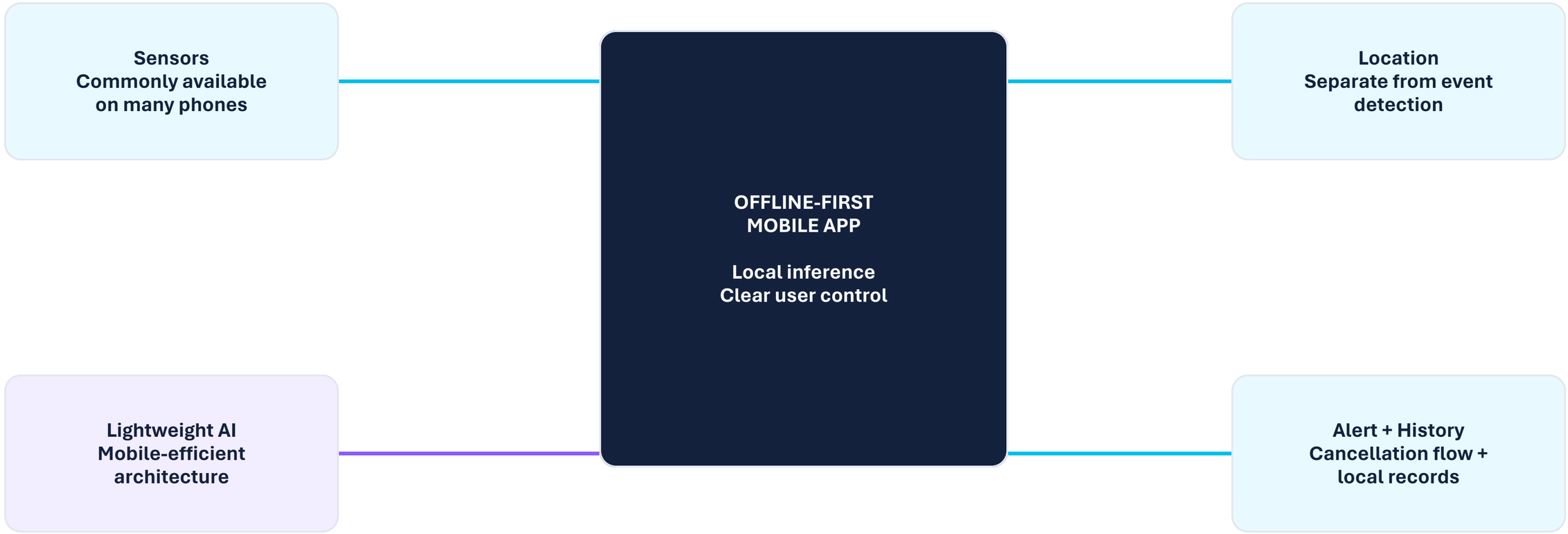
Private, offline classification.
Challenge: limited compute resources.

Comparing Different Approaches

				Proposed Offline-First
Internet dependency	Usually required	Varies	Can be optional	Offline-first
Additional hardware	No	Often yes	No	No
Local processing	Often no	Often yes	Possible	Emphasized
Privacy	Provider-dependent	More controllable	Implementation-dependent	Privacy-conscious
Location support	Supported	Varies	Varies	Captured separately
Mobile suitability	Moderate	Less flexible	High	Designed for mobile

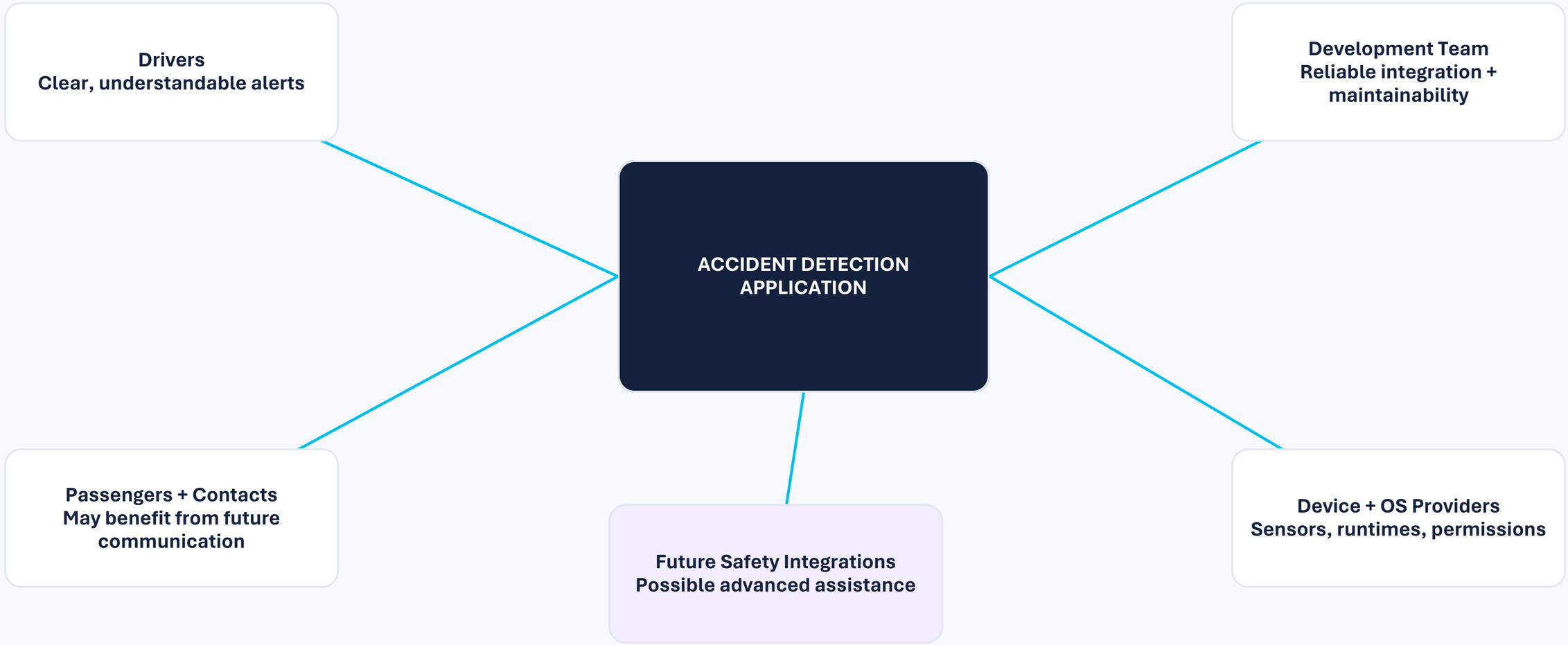
High-level design comparison only — not a controlled benchmark.

What Makes Our Approach Useful?



A pragmatic design contribution — not a claim of universal superiority.

Who Will Use the Application?



Emergency-service integration is a potential future extension, not a guaranteed prototype feature.

Functional Requirements: Sensor and AI

FR-01	Read accelerometer data when available	**HIGH**
FR-02	Read gyroscope data when supported	**HIGH**
FR-03	Preprocess data into model input format	**HIGH**
FR-04	Classify Normal Driving, Near Miss, or Crash	**HIGH**
FR-05	Operate core classification without internet	**HIGH**
FR-06	Handle unavailable sensors, invalid inputs, and inference errors	**MEDIUM**

Functional Requirements: Alerts and Location

FR-07 HIGH

Show possible-crash alert when logic triggers

FR-08 HIGH

Provide countdown and cancellation option

FR-09 MEDIUM

Attempt location capture when available

FR-10 HIGH

Save event details locally

FR-11 MEDIUM

Display saved event history

FR-12 HIGH

Continue without GPS/internet when core is operational



No automatic emergency-contact dispatch is implied.

Non-Functional Requirements

Quality targets for a trustworthy mobile prototype.

Performance

Responsive inference + UI

Reliability

Robust to interruptions

Offline Availability

Core independent of servers

Privacy

Local processing; minimal transfer

Usability

Clear, understandable alerts

Battery Efficiency

Minimize sensor overhead

Compatibility

Selected Android devices

Maintainability

Modular, clear structure

Safety

Possible events—not confirmed accidents

Testing and Success Criteria

A dashboard of measurable checks agreed by the team.

AI EVALUATION

Per-class precision, recall, F1
Confusion matrix
Crash + near-miss recall
False-positive / false-negative review

MOBILE APP

Sensor availability
Model input compatibility
Inference + error handling
Countdown, cancellation, history

OFFLINE + DEVICE

No-internet operation
GPS denied / unavailable
Battery + responsiveness
Selected Android devices

USABILITY

Alert comprehension
Cancellation discoverability
Event-detail readability
Team-defined acceptance tests

SUCCESS = repeatable evidence, reviewed together — not a single headline metric.

Limitations and Future Improvements

CURRENT LIMITATIONS

- Dataset quality and class balance affect performance
- Sensors vary across devices and driving conditions
- Sudden movement can trigger false alarms
- GPS availability and accuracy vary
- Continuous monitoring may affect battery life
- Prototype cannot guarantee every accident is detected
- Calls, SMS, and emergency links need separate development

FUTURE IMPROVEMENTS

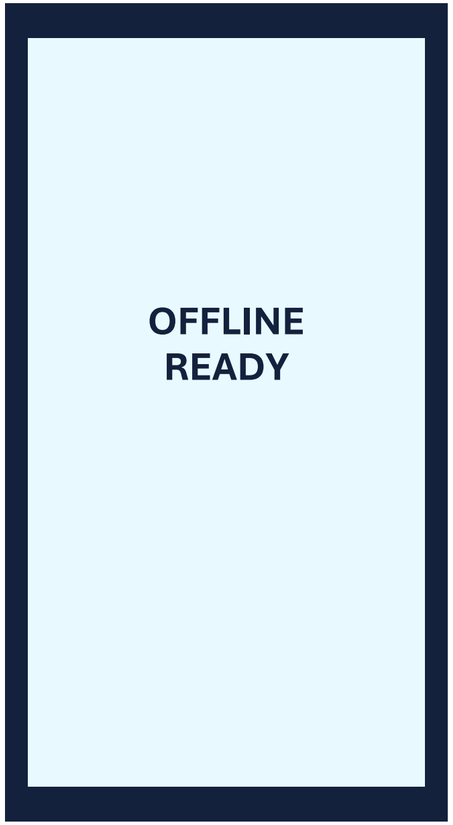
- Collect more diverse, representative sensor data
- Improve near-miss classification
- Validate on physical devices in real conditions
- Optimize model size and battery consumption
- Enable configurable alert behavior
- Explore secure emergency-contact integration
- Expand evaluation and user feedback

Building a practical, privacy-conscious, offline-first safety prototype.

References and Closing

VERIFIED SOURCE FAMILIES

- Official Flutter documentation
- Official TensorFlow and TensorFlow Lite documentation
- Official Android sensor documentation
- Official Android location and permission documentation
- Verified VZCrash dataset page or paper — if confirmed
- Verified peer-reviewed smartphone-sensor studies — if available



Smarter Detection.
On-Device Intelligence.
A More Responsive Safety Experience.

TEAM MEMBERS • CONTACT DETAILS